

ĐẠI HỌC QUỐC GIA TP HCM
TRƯỜNG ĐẠI HỌC KHOA HỌC TỰ NHIÊN
KHOA CÔNG NGHỆ THÔNG TIN

Báo cáo thực hành cá nhân 2

Đề tài: Cài đặt Harris Corner Detection

Môn học: Thị giác máy tính

Sinh viên thực hiện:

Võ Ngọc Hiếu (23122027)

Giáo viên hướng dẫn:

TS. Võ Hoài Việt

ThS. Phạm Minh Hoàng

Ngày 30 tháng 4 năm 2026



Mục lục

1	Giới thiệu	1
2	Cấu trúc thư mục	1
3	Hướng dẫn cài đặt và sử dụng	2
3.1	Yêu cầu hệ thống và Cài đặt	2
3.2	Cấu hình môi trường Visual Studio Code	3
3.3	Chạy chương trình thủ công	3
4	Cơ sở lý thuyết của các thuật toán Harris Corner Detection	3
4.1	Phép tích chập 2 chiều và Xử lý biên	4
4.2	Cơ sở toán học của Thuật toán Harris Corner	4
4.3	Các phương pháp Tối ưu hóa Thuật toán	6
4.3.1	Integer Arithmetic và Gộp chuyển xám:	6
4.3.2	Gộp vòng lặp giảm I/O bộ nhớ:	6
4.3.3	Gaussian phân tách 1D (Separable Gaussian Filter):	7
4.3.4	Kết quả thực nghiệm và Đánh giá hiệu năng	7
5	Kết luận	8
	Tài liệu	9

Danh sách hình vẽ

1	Ảnh ví dụ (Nguồn: wikipedia.org)	4
2	Phát hiện góc với thuật toán Harris cơ bản	6
3	Phát hiện góc với thuật toán Harris cơ bản và sau khi tối ưu	7

1 Giới thiệu

Báo cáo này trình bày về quá trình cài đặt thuật toán Harris Corner Detection thủ công cũng như các phương pháp tối ưu hóa tốc độ thực thi của thuật toán.

- Ngôn ngữ lập trình: **C/C++** kết hợp với thư viện **OpenCV**.
- Môi trường: **Arch Linux**.
- Công cụ: **Visual Studio Code**.

Đồ án tuân thủ yêu cầu không sử dụng các hàm xử lý ảnh có sẵn của thư viện OpenCV (như `cv::cornerHarris`, `cv::Sobel`, `cv::GaussianBlur`). Toàn bộ logic thuật toán được lập trình thủ công thông qua việc thao tác trực tiếp trên con trỏ pixel (`uchar*`, `float*`) và sử dụng số học con trỏ; chỉ sử dụng các hàm I/O và trực quan hóa cơ bản (`cv::imread`, `cv::imwrite`, `cv::imshow`, `cv::circle`).

2 Cấu trúc thư mục

```
/
├── .vscode
│   ├── c_cpp_properties.json
│   ├── launch.json
│   └── tasks.json
├── bin
│   └── 23122027
├── build
├── images
│   ├── input
│   │   └── test.jpg
│   └── output
├── include
│   ├── imgUtils.hpp
│   └── libs.hpp
├── Makefile
├── run.sh
└── src
    ├── imgUtils.cpp
    └── main.cpp
```

3 Hướng dẫn cài đặt và sử dụng

3.1 Yêu cầu hệ thống và Cài đặt

Để biên dịch và chạy chương trình trên môi trường Arch Linux, ta cần cài đặt các công cụ phát triển cơ bản và thư viện tương ứng.

1. **Cập nhật hệ thống và cài đặt các gói cần thiết:** Mở terminal và chạy lệnh sau để cài đặt trình biên dịch (`gcc`, `g++`), trình gỡ lỗi (`gdb`), công cụ build (`make`), công cụ quản lý thư viện (`pkgconf`), và thư viện `opencv`:

```
1 sudo pacman -Syu
2 sudo pacman -S base-devel gdb pkgconf opencv
3
```

2. **Biên dịch chương trình:** Di chuyển terminal vào thư mục gốc của đồ án (nơi chứa file `Makefile`) và thực thi lệnh `make`:

```
1 make
2
```

Quá trình này sẽ sử dụng `g++` cùng cấu hình từ `pkg-config opencv4` để biên dịch mã nguồn trong thư mục `src/` và tạo ra file thực thi `23122027` nằm trong thư mục `bin/`.

3. **Chạy chương trình thủ công:** Sẽ được đề cập ở phần mục sau.
4. **Chạy chương trình tự động:** Kịch bản `run.sh` đã được chuẩn bị sẵn để chạy thử nghiệm tất cả các tính năng của đồ án. Trước khi chạy, đảm bảo bạn đã tạo thư mục `images/input/` và đưa một bức ảnh mẫu có tên `test.jpg` vào đó.

Sau đó, cấp quyền thực thi và chạy script:

```
1 chmod +x run.sh
2 ./run.sh
3
```

Script sẽ tuần tự gọi file thực thi để thực hiện các tác vụ xử lý ảnh và tự động lưu ảnh kết quả vào thư mục `images/output/`.

3.2 Cấu hình môi trường Visual Studio Code

Để thuận tiện cho việc phát triển và gỡ lỗi mã nguồn C++, đồ án đã được cấu hình sẵn cho trình soạn thảo Visual Studio Code thông qua các file trong thư mục `.vscode`:

- **Cấu hình IntelliSense (`c_cpp_properties.json`):** File này khai báo chuẩn ngôn ngữ `gnu++17` và cung cấp đường dẫn đến thư viện OpenCV (`/usr/include/opencv4`). Điều này giúp VS Code nhận diện đúng các hàm của OpenCV, hỗ trợ lập trình trên môi trường Linux.
- **Tác vụ biên dịch (`tasks.json`):** Định nghĩa một tác vụ mang tên `build with make`. Tác vụ này tự động thực thi lệnh `make` đã được định nghĩa trong `Makefile`, giúp quá trình biên dịch (build) dự án diễn ra tự động hóa chỉ với một phím tắt.
- **Cấu hình gỡ lỗi (`launch.json`):** Sử dụng trình gỡ lỗi `gdb` của hệ thống. Cấu hình được thiết lập để tự động gọi tác vụ biên dịch trước (`preLaunchTask`), sau đó khởi chạy file thực thi `23122027` với các đối số dòng lệnh mẫu. Thiết lập này cho phép đặt breakpoint và theo dõi giá trị biến một cách trực quan trong quá trình chạy thử nghiệm.

3.3 Chạy chương trình thủ công

Chương trình yêu cầu truyền tham số qua dòng lệnh để thực thi các phiên bản thuật toán khác nhau.

Chạy thuật toán Harris cơ bản:

```
1 ./bin/23122027 -harris <InputFilePath> <OutputFilePath>
```

Chạy thuật toán Harris tối ưu hóa tốc độ:

```
1 ./bin/23122027 -optHarris <InputFilePath> <OutputFilePath>
```

4 Cơ sở lý thuyết của các thuật toán Harris Corner Detection

Ta sẽ sử dụng ảnh sau như một ví dụ minh họa cho các thao tác xử lý ảnh:



Hình 1: Ảnh ví dụ (Nguồn: [wikipedia.org](https://www.wikipedia.org))

4.1 Phép tích chập 2 chiều và Xử lý biên

Phép tích chập (Convolution) là nền tảng của các bộ lọc không gian. Quá trình này sử dụng một kernel K kích thước $k \times k$ trượt dọc theo ảnh gốc I . Tại vùng biên, chương trình sử dụng kỹ thuật **Replicate Padding** (lặp lại viền ảnh) — pixel nằm ngoài biên được thay thế bằng giá trị pixel biên gần nhất. Công thức tích chập tại tọa độ (x, y) với bán kính $pad = \lfloor k/2 \rfloor$:

$$S(x, y) = \sum_{i=-pad}^{pad} \sum_{j=-pad}^{pad} I(x+i, y+j) \cdot K(-i, -j)$$

4.2 Cơ sở toán học của Thuật toán Harris Corner

Thuật toán Harris dựa trên việc quan sát sự thay đổi cường độ sáng của một cửa sổ nhỏ khi dịch chuyển theo mọi hướng. Hàm năng lượng (sự thay đổi cường độ) khi dịch chuyển cửa sổ một khoảng (u, v) được định nghĩa là:

$$E(u, v) = \sum_{x, y} w(x, y) [I(x+u, y+v) - I(x, y)]^2$$

Trong đó, $w(x, y)$ là hàm cửa sổ trọng số (trong đề án này là Gaussian), I là cường độ sáng. Áp dụng khai triển Taylor bậc nhất cho $I(x+u, y+v)$:

$$I(x+u, y+v) \approx I(x, y) + I_x u + I_y v$$

Với I_x, I_y là đạo hàm theo hướng X và Y (tính bằng kernel Sobel). Thay vào hàm năng lượng, ta có dạng ma trận:

$$E(u, v) \approx \begin{bmatrix} u & v \end{bmatrix} M \begin{bmatrix} u \\ v \end{bmatrix}$$

Trong đó, M là **Ma trận cấu trúc (Structure Tensor)**:

$$M = \sum_{x,y} w(x, y) \begin{bmatrix} I_x^2 & I_x I_y \\ I_x I_y & I_y^2 \end{bmatrix} = \begin{bmatrix} S_{xx} & S_{xy} \\ S_{xy} & S_{yy} \end{bmatrix}$$

Hàm cửa sổ $w(x, y)$ ở đây là bộ lọc **Gaussian 2D**, được định nghĩa:

$$G_{2D}(x, y) = \frac{1}{2\pi\sigma^2} \exp\left(-\frac{x^2 + y^2}{2\sigma^2}\right)$$

Bộ lọc này đảm bảo các pixel gần tâm của sổ đóng góp trọng số lớn hơn các pixel ở rìa, giúp ước lượng ma trận cấu trúc M mượt mà và bền vững hơn trước nhiễu. Các thành phần S_{xx}, S_{yy}, S_{xy} chính là kết quả của việc làm trơn các ma trận bình phương đạo hàm bằng G_{2D} .

Để phân loại điểm đó có phải là góc hay không mà không cần tính trực tiếp trị riêng λ_1, λ_2 của ma trận M , Harris và Stephens đề xuất độ đo hàm phản hồi R :

$$R = \det(M) - k \cdot (\text{trace}(M))^2$$

Với $\det(M) = S_{xx}S_{yy} - S_{xy}^2$, $\text{trace}(M) = S_{xx} + S_{yy}$ và hằng số thực nghiệm $k = 0.04$. Nếu R lớn và dương (λ_1 và λ_2 đều lớn), điểm đó là một góc.

Cuối cùng, thuật toán áp dụng kỹ thuật **Non-Maximum Suppression (NMS)** để loại bỏ các điểm phản hồi yếu và gom cụm. Về cơ chế: vì một góc thực tế trong ảnh thường kích hoạt một *vùng* rộng gồm nhiều pixel liên kề cùng có R cao, NMS chỉ giữ lại đỉnh cực bộ cao nhất và loại toàn bộ các điểm lân cận yếu hơn. Cụ thể, với mỗi pixel (x, y) : nếu $R(x, y)$ vượt ngưỡng $0.01 \cdot R_{\max}$ và là giá trị lớn nhất trong vùng lân cận 3×3 xung quanh nó, pixel đó mới được công nhận là một góc thực sự:

$$R(x, y) > 0.01 \cdot R_{\max} \quad \text{và} \quad R(x, y) = \max_{(i,j) \in \mathcal{N}_{3 \times 3}(x,y)} R(i, j)$$

Ví dụ minh họa kết quả thuật toán Harris:



(a) Ảnh gốc



(b) Điểm góc phát hiện được

Hình 2: Phát hiện góc với thuật toán Harris cơ bản

4.3 Các phương pháp Tối ưu hóa Thuật toán

Thuật toán cơ bản dù chính xác nhưng tiêu tốn rất nhiều chu kỳ xung nhịp CPU do thực hiện tính toán độ phức tạp cao trên từng pixel. Hàm `optHarris` áp dụng các kỹ thuật sau để tăng tốc:

4.3.1 Integer Arithmetic và Gộp chuyển xám:

Thay vì tính toán cường độ xám bằng số phẩy động (float), phương trình xám được nhân với 1000 để sử dụng toàn bộ toán tử nguyên:

$$Y = \frac{R \times 299 + G \times 587 + B \times 114}{1000}$$

Toán tử nguyên trên ALU của CPU thực thi nhanh hơn nhiều so với FPU (đơn vị dấu phẩy động).

4.3.2 Gộp vòng lặp giảm I/O bộ nhớ:

Việc tính đạo hàm Sobel X, Y và Tensor I_{xx}, I_{yy}, I_{xy} yêu cầu phải duyệt qua ma trận ảnh nhiều lần, gây ra hiện tượng cache miss và nghẽn băng thông RAM. Phiên bản tối ưu tính trực tiếp xấp xỉ đạo hàm và bình phương ngay trong một vòng lặp duyệt ảnh duy nhất, giữ dữ liệu nằm trên L1/L2 Cache lâu hơn.

4.3.3 Gaussian phân tách 1D (Separable Gaussian Filter):

Hàm Gaussian 2D $G_{2D}(x, y)$ có **tính chất tách được tuyến tính**:

$$G_{2D}(x, y) = G_{1D}(x) \cdot G_{1D}(y), \quad \text{với } G_{1D}(t) = \frac{1}{\sqrt{2\pi}\sigma} \exp\left(-\frac{t^2}{2\sigma^2}\right)$$

Nhờ đó, thay vì tích chập trực tiếp với kernel 2D kích thước $k \times k$ (đòi hỏi $O(k^2)$ phép nhân cho mỗi pixel), thuật toán tách thành 2 bước tích chập với kernel 1D kích thước k :

- **Quét ngang:** Tích chập từng hàng ảnh với G_{1D} , lưu vào buffer trung gian.
- **Quét dọc:** Tích chập từng cột của buffer với G_{1D} , thu được kết quả cuối.

Điều này làm giảm độ phức tạp tính toán xuống còn $O(2k)$. Với $k = 5$, số phép nhân trên mỗi pixel giảm từ 25 xuống 10, tăng tốc độ làm mượt lên hơn 2 lần.

4.3.4 Kết quả thực nghiệm và Đánh giá hiệu năng

- Thời gian chạy thuật toán Harris cơ bản: **755.618 ms**.
- Thời gian chạy thuật toán Harris tối ưu: **176.521 ms**.



(a) Chưa tối ưu



(b) Đã tối ưu

Hình 3: Phát hiện góc với thuật toán Harris cơ bản và sau khi tối ưu

Đánh giá: Thời gian thực thi của thuật toán đã giảm mạnh. Tốc độ xử lý của phiên bản tối ưu **nh nhanh hơn gấp 4 lần** so với phiên bản cơ bản ban đầu. Kết quả này minh chứng rõ ràng cho sự hiệu quả của các kỹ thuật tối ưu hóa thuật toán đã được phân tích ở trên.

5 Kết luận

Đồ án đã hoàn thành các yêu cầu đề ra:

- Triển khai thành công thuật toán Harris Corner Detection từ đầu bằng thao tác con trỏ cấp thấp trong ngôn ngữ C++.
- Triển khai các phương pháp tối ưu hóa phần cứng, minh chứng được khả năng tư duy tối ưu thuật toán thay vì chỉ gọi hàm thư viện.

Tài liệu

- [1] Rafael C. Gonzalez and Richard E. Woods. *Digital Image Processing*. Pearson, New York, NY, 4th edition, 2018.
- [2] Ana Huamán. OpenCV: Harris corner detector — docs.opencv.org. https://docs.opencv.org/4.x/d4/d7d/tutorial_harris_detector.html. [Accessed 02-04-2026].